

OSSINT Blueprint

OSSINT PRODUCT BLUEPRINT

=====

PIPELINE: identifier -> normalize -> parallel source queries -> correlation graph -> confidence -> report
 PIVOTING: every finding seeds new queries (email -> username -> name -> phone), max_depth=2.

TECHNIQUE MATRIX

- Email: registration enumeration (user-scanner / socialscan), breaches (HIBP, DeHashed), Gravatar JSON, Google-account surface (Epieos/GHunt)
- Phone: E.164 + carrier/line-type (phonenumbers), enrichment (Twilio Lookup, Numverify), WhatsApp/Telegram presence, breach records
- Name: search-engine x-ray (Serper, site: dorks), people-data APIs (PDL/Apollo)
- Username: Mairgret (3000+ sites), Sherlock, Blackbird

BREACHES: HIBP v3 via hibp-api-key (free at haveibeenpwned.com). 200 = breaches found, 404 = clean. Add more engines (DeHashed/Snubase/IntelX) behind the same Source API.

WEB APP (FastAPI + Jinja2)

```
python run_web.py          -> http://127.0.0.1:8000 (opens browser automatically)
POST /search               -> runs the pipeline, saves report, redirects to result page
GET /report/{id}           -> results page (identifiers, findings, confidence, pivots)
GET /report/{id}/pdf       -> server-generated PDF download
Browser Print / Save-as-PDF -> printable layout via print CSS
Reports persist as JSON under reports/ (recent searches listed on the home page).
```

CONFIDENCE MODEL

VERIFIED - target server explicitly confirmed (HTTP 200 profile, reset-flow response)
 LIKELY - corroborated search hit / single source
 UNSURE - collision-prone (esp. names)

ENGINEERING: async httpx + per-source rate limits, proxy rotation, module health-checks (module rot is the #1 failure mode), graph persistence (JSON + GML), Redis caching.

COMPLIANCE: GDPR/CCPA controller duties, FCRA ToS prohibition on employment/credit/tenant screening, platform-ToS review of enumeration modules. For authorized use only.

USAGE

```
pip install -r requirements.txt
pipx install mairgret          # optional username sweep (3000+ sites)
set HIBP_API_KEY=...          # optional breach module
set SERPER_API_KEY=...        # optional search x-ray
python run_web.py              # start the web app
python -m ossint "john.doe@gmail.com" # or CLI mode
python build_pdf.py            # regenerate this document
```

ossint/models.py

```

from __future__ import annotations
from dataclasses import dataclass, field
from enum import Enum
import time

class IdentifierType(str, Enum):
    EMAIL = "email"
    PHONE = "phone"
    USERNAME = "username"
    NAME = "name"
    DOMAIN = "domain"

class Confidence(str, Enum):
    VERIFIED = "verified" # target server explicitly confirmed existence
    LIKELY = "likely" # >=2 independent sources corroborate
    UNSURE = "unsure" # single weak source / collision risk

@dataclass(frozen=True)
class Identifier:
    type: IdentifierType
    value: str

    def key(self) -> str:
        return f"{self.type.value}:{self.value.strip().lower()}"

    def __str__(self) -> str:
        return f"{self.type.value}:{self.value}"

@dataclass
class Finding:
    source: str
    identifier: Identifier
    confidence: Confidence
    url: str = None
    details: dict = field(default_factory=dict)
    pivots: list = field(default_factory=list)
    discovered: float = field(default_factory=time.time)

```

ossint/normalizers.py

```

from __future__ import annotations
import hashlib
import re

import phonenumbers
from phonenumbers import NumberParseException, PhoneNumberFormat

from .models import Identifier, IdentifierType

def normalize_email(raw: str) -> Identifier:
    value = raw.strip().lower()
    if not re.match(r"^[^@\s]+@[^@\s]+\.[^@\s]+$", value):
        raise ValueError(f"invalid email: {raw}")
    return Identifier(IdentifierType.EMAIL, value)

def normalize_phone(raw: str, default_region: str = "US") -> Identifier:
    try:
        num = phonenumbers.parse(raw, default_region)
    except NumberParseException as exc:
        raise ValueError(f"invalid phone: {raw}") from exc
    if not phonenumbers.is_valid_number(num):
        raise ValueError(f"phone not valid: {raw}")
    return Identifier(IdentifierType.PHONE, phonenumbers.format_number(num, PhoneNumberFormat.E164))

def _name_parts(name: str) -> list:
    parts = [re.sub(r"^[a-z0-9]", "", p.lower()) for p in name.split()]
    return [p for p in parts if p]

def username_candidates(name: str) -> list:
    """jdoe, johndoe, john.doe, john-doe, doe.john, j_doe ..."""
    parts = _name_parts(name)
    if not parts:
        return []
    first, last = parts[0], parts[-1]
    cands = {
        first + last, first + "." + last, first + "-" + last,
        first[0] + last, first[0] + "." + last, first + "_" + last,
        last + first, last + "." + last, first, last,
    }
    return sorted(c for c in cands if len(c) >= 3)

def email_permutations(name: str, domains=("gmail.com", "outlook.com", "yahoo.com")) -> list:
    parts = _name_parts(name)
    if len(parts) < 2:
        return []
    f, l = parts[0], parts[-1]
    patterns = {f"{f}.{l}", f"{f}{l}", f"{f[0]}{l}", f"{f[0]}.{l}"}
    return [
        Identifier(IdentifierType.EMAIL, f"{p}@{d}")
        for p in patterns if len(p) >= 4
        for d in domains
    ]

def gravatar_hash(email: str) -> str:
    return hashlib.md5(email.strip().lower().encode()).hexdigest()

```

ossint/graph.py

```

from __future__ import annotations
import networkx as nx

from .models import Finding

class CorrelationGraph:
    def __init__(self):
        self.g = nx.Graph()
        self.findings: list = []

    def add_finding(self, f: Finding) -> None:
        self.findings.append(f)
        src = f"src:{f.source}"
        ident_key = f.identifier.key()
        if not self.g.has_node(ident_key):
            self.g.add_node(ident_key, **{"type": f.identifier.type.value,
                                           "value": f.identifier.value,
                                           "label": "identifier"})

        if not self.g.has_node(src):
            self.g.add_node(src, **{"label": "source"})
        self.g.add_edge(ident_key, src, kind="observed_in", confidence=f.confidence.value)

        for pivot in f.pivots:
            pkey = pivot.key()
            if not self.g.has_node(pkey):
                self.g.add_node(pkey, **{"type": pivot.type.value, "value": pivot.value,
                                           "label": "identifier"})

            # edge weight accumulates corroboration across sources
            if self.g.has_edge(ident_key, pkey):
                self.g[ident_key][pkey]["weight"] += 1
            else:
                self.g.add_edge(ident_key, pkey, kind="co-occurrence", weight=1)

    def score(self, key: str) -> int:
        """Corroboration = sum of edge weights (multiple independent hits = strong)."""
        if key not in self.g:
            return 0
        return sum(d.get("weight", 1) for _, _, d in self.g.edges(key, data=True))

    def save(self, path: str) -> None:
        nx.write_gml(self.g, path)

```

ossint/confidence.py

```
from .models import Confidence, IdentifierType

def classify(finding_count: int, independent_sources: int, ident_type: str) -> Confidence:
    if ident_type == IdentifierType.NAME.value:
        # names are inherently collision-prone: require strong corroboration
        return Confidence.VERIFIED if independent_sources >= 3 else Confidence.LIKELY
    if finding_count >= 1 and independent_sources >= 2:
        return Confidence.VERIFIED
    return Confidence.LIKELY if finding_count else Confidence.UNSURE
```

ossint/reporting.py

```

from __future__ import annotations
from collections import defaultdict

def render_markdown(graph) -> str:
    """Render a human-readable markdown report from the correlation graph."""
    lines = ["# OSSINT Report", ""]
    by_type = defaultdict(list)
    for n, d in graph.g.nodes(data=True):
        if d.get("label") == "identifier":
            by_type[d["type"]].append((n, d))
    for t, nodes in sorted(by_type.items()):
        if not nodes:
            continue
        lines += [f"## {t.upper()} ({len(nodes)})", ""]
        lines += ["| Identifier | Corroboration |", "|---|---|"]
        for key, d in sorted(nodes, key=lambda x: -graph.score(x[0])):
            lines.append(f"| {d['value']} | {graph.score(key)} |")
        lines.append("")
    lines += ["## Findings", ""]
    for u, v, d in graph.g.edges(data=True):
        if d.get("kind") == "observed_in":
            lines.append(f"- {u} observed via {v} [{d['confidence']}]")
    return "\n".join(lines)

def graph_to_dict(graph) -> dict:
    """Serialize the correlation graph + findings into a JSON-safe dict."""
    identifiers = []
    for n, d in graph.g.nodes(data=True):
        if d.get("label") == "identifier":
            identifiers.append({"key": n, "type": d["type"], "value": d["value"],
                               "score": graph.score(n)})

    findings = []
    for f in graph.findings:
        findings.append({
            "source": f.source,
            "identifier": {"key": f.identifier.key(), "type": f.identifier.type.value,
                           "value": f.identifier.value},
            "confidence": f.confidence.value,
            "url": f.url,
            "details": f.details,
        })

    pivots = []
    for u, v, d in graph.g.edges(data=True):
        if d.get("kind") == "co-occurrence":
            pivots.append({"from": u, "to": v, "weight": d.get("weight", 1)})
    return {"identifiers": identifiers, "findings": findings, "pivots": pivots}

```

ossint/orchestrator.py

```

from __future__ import annotations
import asyncio
import logging

import httpx

from .graph import CorrelationGraph
from .models import Identifier, IdentifierType
from .normalizers import username_candidates
from .sources import REGISTRY
from .sources.wrappers import MaignetSource, SocialscanSource
from .sources.dorks import SerperDorkSource
from .sources.telegram_phone import TelegramPhoneSource
from .sources.breaches import HibpSource

log = logging.getLogger("ossint")

ACTIVE_SOURCES = REGISTRY + [MaignetSource(), SocialscanSource(),
                              SerperDorkSource(), TelegramPhoneSource(),
                              HibpSource()]

def expand(identifier: Identifier) -> list:
    """Pivot expansion: identifier -> candidate new identifiers."""
    if identifier.type == IdentifierType.NAME:
        return [Identifier(IdentifierType.USERNAME, u) for u in username_candidates(identifier.value)]
    return []

class Orchestrator:
    def __init__(self, max_depth: int = 2):
        self.graph = CorrelationGraph()
        self.seen: set = set()
        self.max_depth = max_depth

    async def run(self, seed: Identifier) -> CorrelationGraph:
        client = httpx.AsyncClient(http2=True, follow_redirects=True, timeout=30,
                                   headers={"User-Agent": "Mozilla/5.0 (X11; Linux x86_64) ossint/0.1"})
        try:
            await self._wave([seed], 0, client)
        finally:
            await client.aclose()
        return self.graph

    async def _wave(self, batch: list, depth: int, client: httpx.AsyncClient):
        fresh = [i for i in batch if i.key() not in self.seen]
        self.seen.update(i.key() for i in fresh)
        if not fresh or depth > self.max_depth:
            return

        tasks = []
        for i in fresh:
            for s in ACTIVE_SOURCES:
                if s.enabled and i.type in s.handles:
                    tasks.append(asyncio.create_task(s.safe_query(i, client)))

        next_wave: list = []
        if tasks:
            for task in asyncio.as_completed(tasks):
                for f in await task:
                    log.info("hit: %s via %s [%s]", f.identifier, f.source, f.confidence.value)
                    self.graph.add_finding(f)
                    next_wave.extend(f.pivots)
        next_wave += [p for i in fresh for p in expand(i)]
        await self._wave(next_wave, depth + 1, client)

```

ossint/__main__.py

```
import asyncio
import logging
import sys
from pathlib import Path

from .models import Identifier, IdentifierType
from .normalizers import normalize_email, normalize_phone
from .orchestrator import Orchestrator
from .reporting import render_markdown

logging.basicConfig(level=logging.INFO, format="%asctime)s %(levelname)s %(message)s")

def coerce(raw: str) -> Identifier:
    raw = raw.strip()
    if "@" in raw:
        return normalize_email(raw)
    if raw.replace("+", "").isdigit():
        return normalize_phone(raw)
    if any(c in raw for c in " \t"):
        return Identifier(IdentifierType.NAME, raw.title())
    return Identifier(IdentifierType.USERNAME, raw.lower())

def main() -> int:
    if len(sys.argv) < 2:
        print("usage: python -m ossint <email|phone|name|username>")
        return 1
    ident = coerce(" ".join(sys.argv[1:]))
    graph = asyncio.run(Orchestrator(max_depth=2).run(ident))
    out = Path(f"report_{ident.type.value}.md")
    out.write_text(render_markdown(graph))
    graph.save(f"report_{ident.type.value}.gml")
    print(f"[+] report: {out}")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

ossint/sources/base.py

```

from __future__ import annotations
import abc
import asyncio
import logging
import time

import httpx

from ..models import Finding, Identifier

log = logging.getLogger("ossint")

class Source(abc.ABC):
    name: str = "base"
    handles: set = set()
    enabled: bool = True
    min_interval: float = 0.0          # per-source rate limit (seconds)
    _last_call: float = 0.0

    async def rate_limit(self) -> None:
        wait = self.min_interval - (time.monotonic() - self._last_call)
        if wait > 0:
            await asyncio.sleep(wait)
            self._last_call = time.monotonic()

    @abc.abstractmethod
    async def query(self, identifier: Identifier, client: httpx.AsyncClient) -> list:
        ...

    async def safe_query(self, identifier: Identifier, client: httpx.AsyncClient) -> list:
        """Never let one dead source kill the pipeline (module-rot defense)."""
        try:
            await self.rate_limit()
            return await asyncio.wait_for(self.query(identifier, client), timeout=30)
        except asyncio.TimeoutError:
            log.warning("[%s] timeout on %s", self.name, identifier)
        except Exception as exc:
            log.warning("[%s] failed on %s: %s", self.name, identifier, exc)
        return []

```

ossint/sources/native.py

```

from __future__ import annotations

import httpx
import phonenumbers
from phonenumbers import carrier, geocoder, timezone as tz

from ..models import Confidence, Finding, Identifier, IdentifierType
from ..normalizers import gravatar_hash
from .base import Source

class NativeSource(Source):
    """Shared plumbing: verified HTTP checks with confidence=VERIFIED."""

    def finding(self, identifier, url=None, **details):
        return Finding(source=self.name, identifier=identifier,
                       confidence=Confidence.VERIFIED, url=url, details=details)

class GravatarSource(NativeSource):
    name = "gravatar"
    handles = {IdentifierType.EMAIL}

    async def query(self, identifier, client):
        r = await client.get(f"https://en.gravatar.com/{gravatar_hash(identifier.value)}.json")
        if r.status_code != 200:
            return []
        entry = (r.json().get("entry") or [{}])[0]
        if not entry:
            return []
        f = self.finding(identifier, url=f"https://gravatar.com/{entry.get('preferredUsername', '')}",
                        display=entry.get("displayName"))
        if entry.get("preferredUsername"):
            f.pivots.append(Identifier(IdentifierType.USERNAME, entry["preferredUsername"]))
        if entry.get("name", {}).get("formatted"):
            f.pivots.append(Identifier(IdentifierType.NAME, entry["name"]["formatted"]))
        for u in entry.get("urls", []):
            f.details.setdefault("profile_urls", []).append(u.get("value"))
        return [f]

class GithubUserSource(NativeSource):
    name = "github"
    handles = {IdentifierType.USERNAME}

    async def query(self, identifier, client):
        r = await client.get(f"https://api.github.com/users/{identifier.value}")
        if r.status_code == 200:
            data = r.json()
            f = self.finding(identifier, url=data.get("html_url"),
                            created=data.get("created_at"), bio=data.get("bio"))
            if data.get("name"):
                f.pivots.append(Identifier(IdentifierType.NAME, data["name"]))
            if data.get("email"):
                f.pivots.append(Identifier(IdentifierType.EMAIL, data["email"]))
            return [f]
        return []

class RedditUserSource(NativeSource):
    name = "reddit"
    handles = {IdentifierType.USERNAME}

    async def query(self, identifier, client):
        r = await client.get(f"https://www.reddit.com/user/{identifier.value}/about.json",
                            headers={"User-Agent": "ossint/0.1 research"})
        if r.status_code == 200:
            data = r.json().get("data", {})
            return [self.finding(identifier, url=f"https://reddit.com/u/{identifier.value}",
                                created=data.get("created_utc"), karma=data.get("total_karma"))]
        return []

```

```

class TelegramUsernameSource(NativeSource):
    name = "telegram"
    handles = {IdentifierType.USERNAME}

    async def query(self, identifier, client):
        r = await client.get(f"https://t.me/{identifier.value}")
        if r.status_code == 200 and "tgme_page_title" in r.text:
            return [self.finding(identifier, url=f"https://t.me/{identifier.value}")]
        return []

class PhoneMetaSource(NativeSource):
    name = "phone-meta"
    handles = {IdentifierType.PHONE}

    async def query(self, identifier, client):
        num = phonenumbers.parse(identifier.value, None)
        tzfn = getattr(tz, "time_zones_for_number", None) or getattr(tz, "timezones_for_number", None)
        try:
            tzs = str(tzfn(num))
        except Exception:
            tzs = "n/a"
        f = self.finding(identifier,
                        country=geocoder.description_for_number(num, "en"),
                        carrier=carrier.name_for_number(num, "en"),
                        timezones=tsz,
                        line_type=str(phonenumbers.number_type(num)))
        return [f]

REGISTRY = [GravatarSource(), GithubUserSource(), RedditUserSource(),
            TelegramUsernameSource(), PhoneMetaSource()]

```

ossint/sources/wrappers.py

```

from __future__ import annotations
import asyncio
import json
import shutil

import httpx

from ..models import Confidence, Finding, Identifier, IdentifierType
from .base import Source

class MaigretSource(Source):
    """Username sweep across 3000+ sites via the `maigret` CLI (pipx install maigret)."""
    name = "maigret"
    handles = {IdentifierType.USERNAME}

    async def query(self, identifier: Identifier, client: httpx.AsyncClient):
        if not shutil.which("maigret"):
            return []
        proc = await asyncio.create_subprocess_exec(
            "maigret", identifier.value, "--json", "-", "--timeout", "15",
            stdout=asyncio.subprocess.PIPE, stderr=asyncio.subprocess.DEVNULL)
        out, _ = await asyncio.wait_for(proc.communicate(), timeout=180)
        try:
            data = json.loads(out.decode() or "{}")
        except json.JSONDecodeError:
            return []
        findings = []
        for site, info in data.items():
            status = info.get("status", {})
            if status.get("status") == "Exist":
                f = Finding(source=f"maigret:{site}", identifier=identifier,
                           confidence=Confidence.VERIFIED,
                           url=status.get("url"), details=info)
                findings.append(f)
        return findings

class SocialscanSource(Source):
    """Email+username availability across ~15 major platforms via `socialscan`."""
    name = "socialscan"
    handles = {IdentifierType.EMAIL, IdentifierType.USERNAME}

    async def query(self, identifier: Identifier, client: httpx.AsyncClient):
        if not shutil.which("socialscan"):
            return []
        proc = await asyncio.create_subprocess_exec(
            "socialscan", identifier.value, "--json",
            stdout=asyncio.subprocess.PIPE, stderr=asyncio.subprocess.DEVNULL)
        out, _ = await asyncio.wait_for(proc.communicate(), timeout=120)
        try:
            data = json.loads(out.decode() or "[]")
        except json.JSONDecodeError:
            return []
        results = data if isinstance(data, list) else data.get("results", [])
        return [Finding(source=f"socialscan:{r.get('platform')}", identifier=identifier,
                       confidence=Confidence.VERIFIED if r.get("registered") else Confidence.UNSURE,
                       url=r.get("profile_url"), details=r)
                for r in results if r.get("registered")]

```

ossint/sources/dorks.py

```

from __future__ import annotations
import os

import httpx

from ..models import Confidence, Finding, Identifier, IdentifierType
from .base import Source

SITES = ["linkedin.com/in", "x.com", "instagram.com", "facebook.com",
         "tiktok.com/@", "github.com", "medium.com/@", "pinterest.com"]

DORK = {
    IdentifierType.NAME: 'site:{site} "{q}"',
    IdentifierType.USERNAME: 'site:{site} "{q}"',
    IdentifierType.EMAIL: '"{q}" (site:linkedin.com OR site:x.com OR site:facebook.com)',
    IdentifierType.PHONE: '"{q}" (site:linkedin.com OR site:facebook.com)',
}

class SerperDorkSource(Source):
    """Search-engine x-ray. Needs SERPER_API_KEY (serper.dev, generous free tier)."""
    name = "serper-dorks"
    handles = {IdentifierType.NAME, IdentifierType.USERNAME,
                IdentifierType.EMAIL, IdentifierType.PHONE}
    min_interval = 1.0

    async def query(self, identifier: Identifier, client: httpx.AsyncClient):
        key = os.getenv("SERPER_API_KEY")
        if not key:
            return []
        findings = []
        if identifier.type in (IdentifierType.NAME, IdentifierType.USERNAME):
            queries = [DORK[identifier.type].format(site=s, q=identifier.value) for s in SITES]
        else:
            queries = [DORK[identifier.type].format(q=identifier.value)]
        for q in queries:
            r = await client.post("https://google.serper.dev/search",
                                  headers={"X-API-KEY": key, "Content-Type": "application/json"},
                                  json={"q": q, "num": 5})
            if r.status_code != 200:
                continue
            for hit in r.json().get("organic", []):
                findings.append(Finding(
                    source="dork", identifier=identifier,
                    confidence=Confidence.LIKELY, # search-engine hit = corroborate, don't trust
                    url=hit.get("link"),
                    details={"title": hit.get("title"), "snippet": hit.get("snippet")}))
        return findings

```

ossint/sources/telegram_phone.py

```

from __future__ import annotations
import os

from ..models import Confidence, Finding, Identifier, IdentifierType
from .base import Source

class TelegramPhoneSource(Source):
    """Phone -> Telegram existence via contact import. Requires TELEGRAM_API_ID and
    TELEGRAM_API_HASH env vars (my.telegram.org) and a one-time interactive login.
    Telethon is imported lazily so the core pipeline works without it installed."""
    name = "telegram-phone"
    handles = {IdentifierType.PHONE}

    async def query(self, identifier: Identifier, client):
        api_id, api_hash = os.getenv("TELEGRAM_API_ID"), os.getenv("TELEGRAM_API_HASH")
        if not (api_id and api_hash):
            return []
        from telethon import TelegramClient, functions, types # optional dep
        async with TelegramClient("ossint_session", int(api_id), api_hash) as tg:
            result = await tg(functions.contacts.ImportContactsRequest(
                contacts=[types.InputPhoneContact(client_id=0, phone=identifier.value,
                                                    first_name="", last_name=")])
            if result.imported_contacts:
                c = result.imported_contacts[0]
                return [Finding(source="telegram", identifier=identifier,
                                confidence=Confidence.VERIFIED,
                                url=f"https://t.me/{identifier.value.lstrip('+')}",
                                details={"user_id": str(c.user_id)})]
        return []

```

ossint/sources/breaches.py

```
from __future__ import annotations
import os
from urllib.parse import quote

from ..models import Confidence, Finding, Identifier, IdentifierType
from .base import Source

class HibpSource(Source):
    """Have I Been Pwned breach exposure. Needs HIBP_API_KEY (free at haveibeenpwned.com).
    HTTP 200 = email appears in known breaches; 404 = no public breaches."""
    name = "hibp"
    handles = {IdentifierType.EMAIL}
    min_interval = 2.0

    async def query(self, identifier: Identifier, client):
        key = os.getenv("HIBP_API_KEY")
        if not key:
            return []
        url = f"https://haveibeenpwned.com/api/v3/breachedaccount/{quote(identifier.value)}"
        headers = {"hibp-api-key": key, "User-Agent": "ossint/0.1"}
        r = await client.get(url, headers=headers,
                             params={"truncateResponse": "true",
                                     "includeUnverified": "false"})

        if r.status_code == 200:
            names = r.json()
            return [Finding(source="hibp", identifier=identifier,
                           confidence=Confidence.VERIFIED,
                           url="https://haveibeenpwned.com",
                           details={"breaches": names})]

        if r.status_code == 429:
            return [] # HIBP rate limit: skip quietly, next run picks it up
        return []
```

run_web.py

```
import threading
import webbrowser

import uvicorn

if __name__ == "__main__":
    threading.Timer(1.2, lambda: webbrowser.open("http://127.0.0.1:8000")).start()
    print("[+] OSSINT Web running at http://127.0.0.1:8000 (Ctrl+C to stop)")
    uvicorn.run("webapp.server:app", host="127.0.0.1", port=8000, log_level="info")
```

webapp/server.py

```

from __future__ import annotations
import logging
import time
from pathlib import Path

from fastapi import FastAPI, Form, Request
from fastapi.responses import HTMLResponse, RedirectResponse, Response
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates

from ossint.__main__ import coerce
from ossint.orchestrator import Orchestrator

from .pdf_export import render_report_pdf
from .store import ReportStore

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")

BASE = Path(__file__).resolve().parent
ROOT = BASE.parent

app = FastAPI(title="OSSINT Web", docs_url=None, redoc_url=None)
templates = Jinja2Templates(directory=str(BASE / "templates"))
app.mount("/static", StaticFiles(directory=str(BASE / "static")), name="static")
store = ReportStore(ROOT / "reports")

ORDER = ["email", "phone", "username", "name", "domain"]

def prepare_report(data: dict) -> dict:
    """Turn a stored report dict into template-ready view data."""
    graph = data["graph"]
    lookup = {i["key"]: i for i in graph["identifiers"]}

    groups: dict = {}
    for i in graph["identifiers"]:
        groups.setdefault(i["type"], []).append(i)
    groups = {k: sorted(v, key=lambda x: -x["score"]) for k, v in groups.items()}
    groups = {k: groups[k] for k in sorted(groups, key=lambda t: ORDER.index(t) if t in ORDER else 99)}

    findings = []
    for f in graph["findings"]:
        details = f.get("details") or {}
        breaches = details.get("breaches") if isinstance(details, dict) else None
        summary = ", ".join(str(b) for b in breaches[:8]) if isinstance(breaches, list) else ""
        findings.append({
            "identifier": f["identifier"]["value"],
            "source": f["source"],
            "confidence": f["confidence"],
            "url": f.get("url"),
            "details_summary": summary,
        })
    findings.sort(key=lambda x: ("verified", "likely", "unsure").index(x["confidence"])
                          if x["confidence"] in ("verified", "likely", "unsure") else 3)

    pivots = []
    for p in graph["pivots"]:
        a = lookup.get(p["from"])
        b = lookup.get(p["to"])
        pivots.append({"from": a["value"] if a else p["from"],
                       "to": b["value"] if b else p["to"],
                       "weight": p["weight"]})

    return {
        "id": data["id"],
        "seed": data["seed"],
        "when": time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(data["created"])),
        "groups": groups,
        "findings": findings,
        "total": len(findings),
        "shows_details": any(f.get("details_summary") for f in findings),
        "pivots": pivots,
    }

```

```

@app.get("/", response_class=HTMLResponse)
async def index(request: Request):
    return templates.TemplateResponse(request=request, name="index.html",
                                      context={"request": request,
                                              "recent": store.list_recent()})

@app.post("/search")
async def search(q: str = Form(...)):
    q = q.strip()
    try:
        ident = coerce(q)
    except ValueError as exc:
        return HTMLResponse(f"<p>Invalid input: {exc}</p><p><a href='/'>Back</a></p>",
                           status_code=400)
    graph = await Orchestrator(max_depth=2).run(ident)
    rid = store.create(ident, graph)
    return RedirectResponse(f"/report/{rid}", status_code=303)

@app.get("/report/{rid}", response_class=HTMLResponse)
async def report(request: Request, rid: str):
    data = store.load(rid)
    if not data:
        return HTMLResponse("<p>Report not found.</p><p><a href='/'>Back</a></p>", status_code=404)
    return templates.TemplateResponse(request=request, name="report.html",
                                      context={"request": request, **prepare_report(data)})

@app.get("/report/{rid}/pdf")
async def report_pdf(rid: str):
    data = store.load(rid)
    if not data:
        return Response("Report not found", status_code=404)
    prepared = prepare_report(data)
    pdf_bytes = render_report_pdf(prepared)
    return Response(content=pdf_bytes, media_type="application/pdf",
                   headers={"Content-Disposition":
                           f'attachment; filename="ossint_report_{rid}.pdf"'})

```

webapp/store.py

```

from __future__ import annotations
import json
import time
import uuid
from pathlib import Path

from ossint.reporting import graph_to_dict

class ReportStore:
    """Persists each search's correlation graph as JSON in <project>/reports/."""

    def __init__(self, directory: Path):
        self.directory = directory
        self.directory.mkdir(parents=True, exist_ok=True)

    def create(self, ident, graph) -> str:
        rid = uuid.uuid4().hex[:12]
        data = {"id": rid, "seed": ident.key(), "created": time.time(),
               "graph": graph_to_dict(graph)}
        (self.directory / f"{rid}.json").write_text(
            json.dumps(data, indent=2, default=str), encoding="utf-8")
        return rid

    def load(self, rid: str):
        p = self.directory / f"{rid}.json"
        if not p.exists():
            return None
        return json.loads(p.read_text(encoding="utf-8"))

    def list_recent(self, limit: int = 10) -> list:
        items = []
        for p in sorted(self.directory.glob("*.json"),
                       key=lambda f: f.stat().st_mtime, reverse=True)[:limit]:
            try:
                data = json.loads(p.read_text(encoding="utf-8"))
            except Exception:
                continue
            items.append({"id": data["id"], "seed": data["seed"],
                        "when": time.strftime("%Y-%m-%d %H:%M",
                                              time.localtime(data["created"]))})

        return items

```

webapp/pdf_export.py

```

from __future__ import annotations

from fpdf import FPDF

ORDER = ["email", "phone", "username", "name", "domain"]

def _safe(value) -> str:
    """cp1252-safe text so non-ASCII names don't crash the core-font PDF."""
    return (str(value) if value is not None else "").encode("cp1252", "replace").decode("cp1252")

def render_report_pdf(prepared: dict) -> bytes:
    pdf = FPDF()
    pdf.set_auto_page_break(True, margin=15)
    pdf.set_margins(15, 15, 15)
    pdf.add_page()

    pdf.set_font("Helvetica", "B", 20)
    pdf.cell(0, 10, "OSSINT Report", new_x="LMARGIN", new_y="NEXT")
    pdf.set_font("Helvetica", "", 10)
    pdf.set_text_color(90, 90, 90)
    pdf.cell(0, 6, f"Seed: {_safe(prepared['seed'])}", new_x="LMARGIN", new_y="NEXT")
    pdf.cell(0, 6, f"Generated: {_safe(prepared['when'])}", new_x="LMARGIN", new_y="NEXT")
    pdf.set_text_color(0, 0, 0)
    pdf.ln(4)

    groups = prepared["groups"]
    for t in ORDER:
        items = groups.get(t)
        if not items:
            continue
        pdf.set_font("Helvetica", "B", 12)
        pdf.cell(0, 8, f"{t.upper()} ({len(items)}", new_x="LMARGIN", new_y="NEXT")
        pdf.set_font("Helvetica", "", 10)
        for it in items:
            pdf.cell(0, 6, f"  {_safe(it['value'])}  (corroboration {it['score']})",
                      new_x="LMARGIN", new_y="NEXT")
        pdf.ln(3)

    pdf.set_font("Helvetica", "B", 12)
    pdf.cell(0, 8, f"FINDINGS ({len(prepared['findings'])})", new_x="LMARGIN", new_y="NEXT")

    headers = ["Identifier", "Source", "Confidence", "Link", "Details"]
    widths = [50, 40, 28, 28, 44]
    pdf.set_font("Helvetica", "B", 9)
    pdf.set_fill_color(230, 230, 230)
    for h, w in zip(headers, widths):
        pdf.cell(w, 7, h, border=1, fill=True)
    pdf.ln()

    pdf.set_font("Helvetica", "", 9)
    for f in prepared["findings"]:
        row = [_safe(f["identifier"]), _safe(f["source"]), _safe(f["confidence"]),
              _safe(f.get("url") or "-"), _safe(f.get("details_summary") or "-")]
        for c, w in zip(row, widths):
            pdf.cell(w, 7, c[:70], border=1)
        pdf.ln()
        if pdf.get_y() > 260:
            pdf.add_page()

    return bytes(pdf.output())

```

webapp/templates/index.html

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>OSSINT ? People Search</title>
<link rel="stylesheet" href="/static/style.css">
</head>
<body>
<main class="container">
  <header class="hero">
    <h1>OSSINT</h1>
    <p class="tagline">Find every public trace of a name, email, phone number or username.</p>
  </header>
  <form class="search-box" method="post" action="/search">
    <input type="text" name="q" placeholder='e.g. "jane.doe@gmail.com", "+1 415 555 0123", "Jane Doe", "jdoe"' autofocus required>
    <button type="submit">Search</button>
  </form>
  {% if recent %}
  <section class="card">
    <h2>Recent searches</h2>
    <ul class="recent">
      {% for r in recent %}
      <li><a href="/report/{{ r.id }}">{{ r.seed }}</a> <span class="muted">{{ r.when }}</span></li>
      {% endfor %}
    </ul>
  </section>
  {% endif %}
  <footer class="muted">For authorized investigations only.</footer>
</main>
</body>
</html>
```

webapp/templates/report.html

```

<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>OSSINT Report ? {{ seed }}</title>
<link rel="stylesheet" href="/static/style.css">
</head>
<body>
<div class="topbar no-print">
  <a class="brand" href="/">OSSINT</a>
  <div class="actions">
    <button onclick="window.print()">&#128424; Print / Save as PDF</button>
    <a class="btn" href="/report/{{ id }}/pdf">&#11015; Download PDF</a>
  </div>
</div>

<main class="container report">
  <header class="report-header">
    <h1>OSSINT Report</h1>
    <p class="seed">Seed: <code>{{ seed }}</code></p>
    <p class="muted">Generated {{ when }} &#183; {{ total }} finding{% if total != 1 %}s{% endif %}</p>
  </header>

  {% for type, items in groups.items() %}
  <section class="card">
    <h2>{{ type|upper }} ({{ items|length }})</h2>
    <table>
      <thead><tr><th>Identifier</th><th>Corroboration</th></tr></thead>
      <tbody>
        {% for it in items %}
        <tr><td><code>{{ it.value }}</code></td><td>{{ it.score }}</td></tr>
        {% endfor %}
      </tbody>
    </table>
  </section>
  {% endfor %}

  <section class="card">
    <h2>Findings</h2>
    {% if findings %}
    <table>
      <thead>
        <tr><th>Identifier</th><th>Source</th><th>Confidence</th><th>Link</th><th>Details</th><th>Details</th><th>Details</th></tr>
      </thead>
      <tbody>
        {% for f in findings %}
        <tr>
          <td><code>{{ f.identifier }}</code></td>
          <td>{{ f.source }}</td>
          <td><span class="badge conf-{{ f.confidence }}">{{ f.confidence }}</span></td>
          <td>{% if f.url %}<a href="{{ f.url }}" target="_blank" rel="noopener">open</a>{% else %}&mdash;{% endif %}</td>
          <td>{% if f.details_summary %}<td class="muted">{{ f.details_summary }}</td>{% endif %}
        </tr>
        {% endfor %}
      </tbody>
    </table>
    {% else %}
    <p class="muted">No findings for this identifier. Try a different input, or add more sources (HIBP_API_KEY, SERPER_API_KEY, maigre
    {% endif %}
  </section>

  {% if pivots %}
  <section class="card">
    <h2>Pivots (co-occurrences)</h2>
    <ul class="pivots">
      {% for p in pivots %}
      <li><code>{{ p.from }}</code> &#594; <code>{{ p.to }}</code> <span class="muted">(weight {{ p.weight }})</span></li>
      {% endfor %}
    </ul>
  </section>
  {% endif %}

```

```
<footer class="muted no-print">For authorized investigations only.</footer>  
</main>  
</body>  
</html>
```

webapp/static/style.css

```

:root {
  --bg: #0f1220; --card: #171b2e; --line: #262b45;
  --text: #e8eaf2; --muted: #8b91ad;
  --accent: #4f7cff; --ok: #2ecc71; --warn: #f1c40f; --weak: #8b8fa8;
}
* { box-sizing: border-box; }
body { margin: 0; font-family: "Segoe UI", system-ui, sans-serif; background: var(--bg); color: var(--text); }
.container { max-width: 920px; margin: 0 auto; padding: 24px; }
.topbar { display: flex; justify-content: space-between; align-items: center; padding: 10px 24px; background: var(--card); border-bottom: 1px solid var(--line); }
.brand { color: var(--text); text-decoration: none; font-weight: 700; letter-spacing: 1px; }
.actions { display: flex; gap: 8px; }
.btn, button { background: var(--accent); color: #fff; border: 0; border-radius: 8px; padding: 8px 14px; font-size: 14px; cursor: pointer; }
.btn:hover, button:hover { filter: brightness(1.1); }
.hero { text-align: center; padding: 60px 0 30px; }
.hero h1 { font-size: 44px; letter-spacing: 5px; margin: 0; }
.tagline { color: var(--muted); }
.search-box { display: flex; gap: 8px; max-width: 660px; margin: 0 auto; }
.search-box input { flex: 1; padding: 14px 16px; border-radius: 10px; border: 1px solid var(--line); background: #0b0e1b; color: var(--text); }
.search-box input:focus { outline: 2px solid var(--accent); }
.card { background: var(--card); border: 1px solid var(--line); border-radius: 12px; padding: 18px 20px; margin: 18px 0; }
.card h2 { margin-top: 0; font-size: 13px; text-transform: uppercase; letter-spacing: 1px; color: var(--muted); }
table { width: 100%; border-collapse: collapse; font-size: 14px; }
th, td { text-align: left; padding: 8px 10px; border-bottom: 1px solid var(--line); vertical-align: top; }
th { color: var(--muted); font-weight: 600; }
a { color: var(--accent); }
code { background: #0b0e1b; padding: 2px 6px; border-radius: 6px; font-size: 13px; }
.badge { padding: 3px 9px; border-radius: 999px; font-size: 12px; font-weight: 600; text-transform: capitalize; display: inline-block; }
.conf-verified { background: rgba(46,204,113,.15); color: var(--ok); }
.conf-likely { background: rgba(241,196,15,.15); color: var(--warn); }
.conf-unsure { background: rgba(139,143,168,.15); color: var(--weak); }
.muted { color: var(--muted); font-size: 13px; }
.seed code { font-size: 15px; }
.pivots { margin: 0; padding-left: 18px; line-height: 1.8; }
.recent { list-style: none; padding: 0; }
.recent li { padding: 6px 0; border-bottom: 1px solid var(--line); }
.report-header { text-align: center; padding: 20px 0 10px; }
footer { text-align: center; padding: 20px 0; }
@media print {
  .no-print { display: none !important; }
  body { background: #fff; color: #000; }
  .card { background: #fff; border: 1px solid #ccc; break-inside: avoid; }
  code { background: #f0f0f0; color: #000; }
  a { color: #000; text-decoration: none; }
  .badge { border: 1px solid #999; }
  .conf-verified, .conf-likely, .conf-unsure { background: #fff; }
  table, th, td { border-color: #ccc; }
}

```

requirements.txt

```
fastapi
uvicorn[standard]
jinja2
python-multipart
httpx[http2]
phonenumbers
networkx
beautifulsoup4
fpdf2
# optional (installed separately with pip/pipx):
#   pipx install maigret    (username sweep, 3000+ sites)
#   pip install socialscan (fast email/username availability)
#   pip install telethon   (phone -> Telegram existence)
# env vars: HIBP_API_KEY, SERPER_API_KEY, TELEGRAM_API_ID, TELEGRAM_API_HASH
```

build_pdf.py

```

"""python build_pdf.py -> OSSINT_Blueprint.pdf (blueprint + all source files)"""
from pathlib import Path

from fpdf import FPDF

CORE = ["ossint/models.py", "ossint/normalizers.py", "ossint/graph.py",
        "ossint/confidence.py", "ossint/reporting.py", "ossint/orchestrator.py",
        "ossint/__main__.py"]
SRC = ["ossint/sources/base.py", "ossint/sources/native.py",
        "ossint/sources/wrappers.py", "ossint/sources/dorks.py",
        "ossint/sources/telegram_phone.py", "ossint/sources/breaches.py"]
WEB = ["run_web.py", "webapp/server.py", "webapp/store.py", "webapp/pdf_export.py",
        "webapp/templates/index.html", "webapp/templates/report.html",
        "webapp/static/style.css"]

BLUEPRINT = """OSSINT PRODUCT BLUEPRINT
=====

PIPELINE: identifier -> normalize -> parallel source queries -> correlation graph -> confidence -> report
PIVOTING: every finding seeds new queries (email -> username -> name -> phone), max_depth=2.

TECHNIQUE MATRIX
- Email: registration enumeration (user-scanner / socialscan), breaches (HIBP, DeHashed),
  Gravatar JSON, Google-account surface (Epieos/GHunt)
- Phone: E.164 + carrier/line-type (phonenumber), enrichment (Twilio Lookup, Numverify),
  WhatsApp/Telegram presence, breach records
- Name: search-engine x-ray (Serper, site: dorks), people-data APIs (PDL/Apollo)
- Username: Mairgret (3000+ sites), Sherlock, Blackbird

BREACHES: HIBP v3 via hibp-api-key (free at haveibeenpwned.com). 200 = breaches found,
404 = clean. Add more engines (DeHashed/Snubase/IntelX) behind the same Source API.

WEB APP (FastAPI + Jinja2)
python run_web.py          -> http://127.0.0.1:8000 (opens browser automatically)
POST /search               -> runs the pipeline, saves report, redirects to result page
GET /report/{id}           -> results page (identifiers, findings, confidence, pivots)
GET /report/{id}/pdf       -> server-generated PDF download
Browser Print / Save-as-PDF -> printable layout via print CSS
Reports persist as JSON under reports/ (recent searches listed on the home page).

CONFIDENCE MODEL
VERIFIED - target server explicitly confirmed (HTTP 200 profile, reset-flow response)
LIKELY    - corroborated search hit / single source
UNSURE    - collision-prone (esp. names)

ENGINEERING: async httpx + per-source rate limits, proxy rotation, module health-checks
(module rot is the #1 failure mode), graph persistence (JSON + GML), Redis caching.

COMPLIANCE: GDPR/CCPA controller duties, FCRA ToS prohibition on employment/credit/
tenant screening, platform-ToS review of enumeration modules. For authorized use only.

USAGE
pip install -r requirements.txt
pipx install mairgret          # optional username sweep (3000+ sites)
set HIBP_API_KEY=...          # optional breach module
set SERPER_API_KEY=...        # optional search x-ray
python run_web.py              # start the web app
python -m ossint "john.doe@gmail.com" # or CLI mode
python build_pdf.py            # regenerate this document
"""

HEADER = "OSSINT Blueprint - privileged/internal"

class Doc(FPDF):
    def header(self):
        self.set_font("Courier", size=7)
        self.cell(0, 4, HEADER, align="C", new_x="LMARGIN", new_y="NEXT")
        self.ln(2)

def code_block(pdf: FPDF, text: str):
    pdf.set_font("Courier", size=7)

```

```

pdf.set_fill_color(245, 245, 245)
for line in text.splitlines():
    if pdf.get_y() > 270:
        pdf.add_page()
    safe = line[:150].encode("latin-1", "replace").decode("latin-1")
    pdf.cell(0, 3.4, safe, fill=True, new_x="LMARGIN", new_y="NEXT")

def main() -> None:
    pdf = Doc(unit="mm", format="A4")
    pdf.set_auto_page_break(True, margin=15)
    pdf.add_page()
    pdf.set_font("Helvetica", "B", 16)
    pdf.cell(0, 10, "OSSINT Blueprint", new_x="LMARGIN", new_y="NEXT")
    pdf.ln(2)
    code_block(pdf, BLUEPRINT)

    for path in CORE + SRC + WEB + ["requirements.txt", "build_pdf.py"]:
        p = Path(path)
        if not p.exists():
            continue
        pdf.add_page()
        pdf.set_font("Helvetica", "B", 11)
        pdf.cell(0, 6, path.encode("latin-1", "replace").decode("latin-1"), new_x="LMARGIN", new_y="NEXT")
        pdf.ln(1)
        code_block(pdf, p.read_text(encoding="utf-8"))

    pdf.output("OSSINT_Blueprint.pdf")
    print("[+] wrote OSSINT_Blueprint.pdf")

if __name__ == "__main__":
    main()

```